Introduction to Data Visualization

Agenda

- Introduction to Data Visualization
  - Introduction to Matplotlib
  - Line Plot
  - Scatter Plot
  - Bar Plot
  - Box Plot
  - Radar Chart (Spider Chart)
  - Area Plot
  - Polar Plot
  - Tree map
  - Pie Chart
  - Matplotlib for 3D Visualization
    - Key Features and Capabilities

Introduction to Data Visualization

Data visualization is the graphical representation of data to reveal patterns, trends, and insights that might not be easily apparent from raw data.

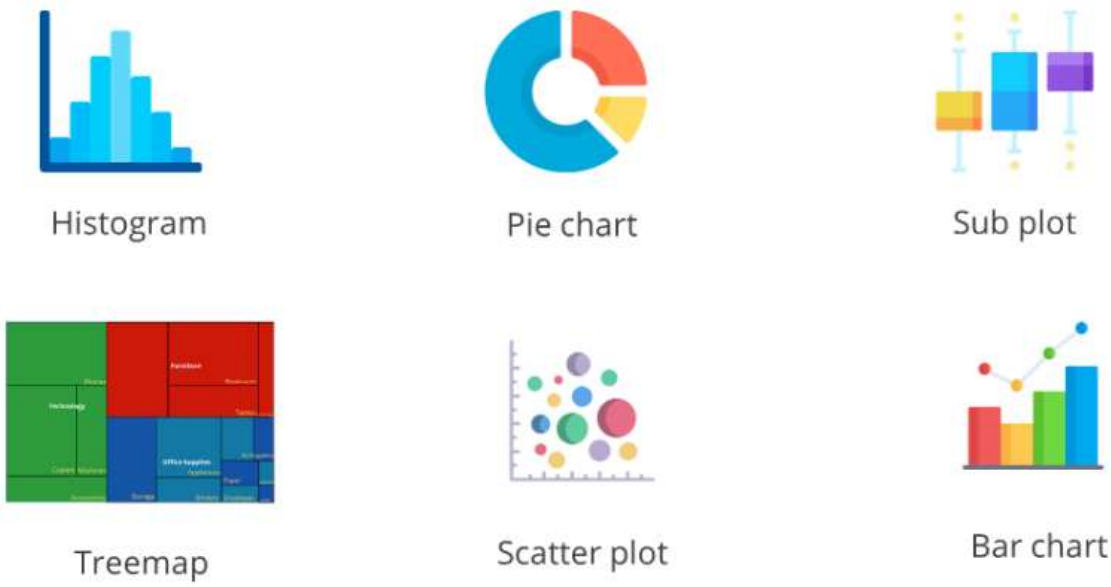
- It involves creating visual elements such as charts, graphs, and maps to communicate complex information in an understandable and interpretable form.

Data visualization tools and libraries, such as Matplotlib, Seaborn, and Plotly, empower analysts, scientists, and business professionals to create compelling visualizations that enhance the understanding of data and support evidence-based decision-making.

Introduction to Matplotlib

Matplotlib is a comprehensive and widely-used data visualization library in Python. It provides a flexible platform for creating static, animated, and interactive visualizations in Python.

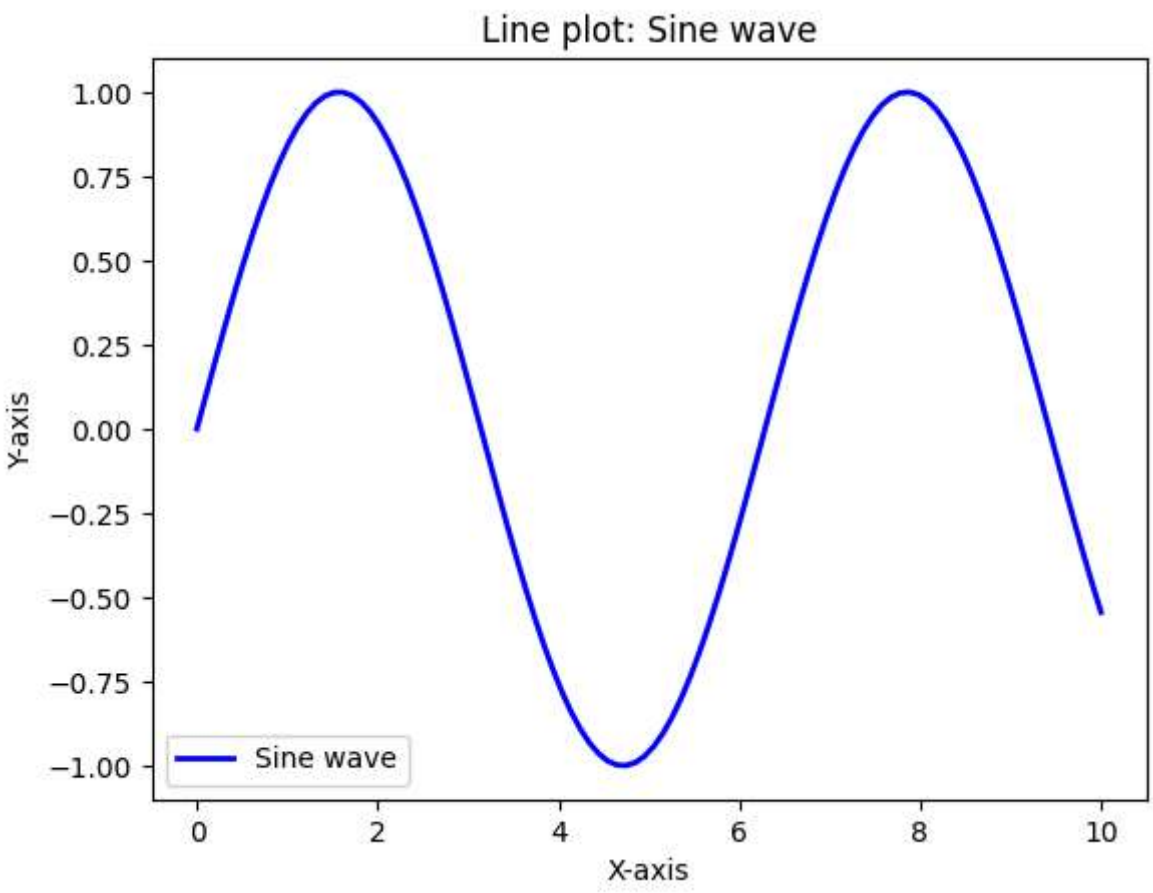
- Matplotlib can plot a variety of graphs, which helps in drawing:



Line Plot:

A line plot is a basic type of chart that displays information as a series of data points connected by straight line segments. It is particularly useful for showing trends over a continuous interval or time.

```
import matplotlib.pyplot as plt
import numpy as np

# Generate data for a line plot
x = np.linspace(0, 10, 100)
y = np.sin(x)

# Create a line plot
plt.plot(x, y, label='Sine wave', color='blue', linestyle='-', linewidth=2)

# Add labels and title
plt.xlabel('X-axis')
plt.ylabel('Y-axis')
plt.title('Line plot: Sine wave')

# Add a legend
plt.legend()

# Show the plot
plt.show()
```

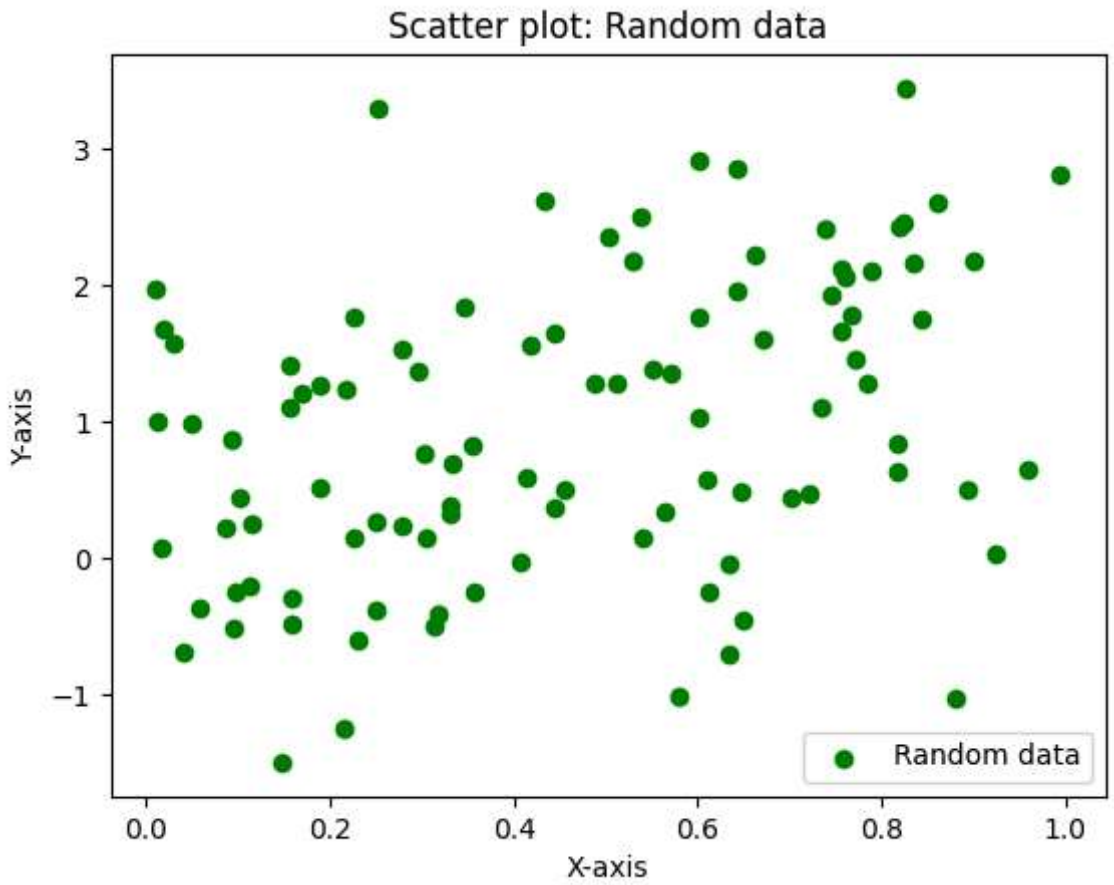


Scatter Plot:

A scatter plot is used to display the relationship between two continuous variables.

- Each point on the plot represents an observation, and the position of the point is determined by the values of the two variables.
- It is helpful for identifying patterns, clusters, or trends in the data.

```
import matplotlib.pyplot as plt
import numpy as np

# Generate data for a scatter plot
x = np.random.rand(100)
y = 2 * x + np.random.randn(100)

# Create a scatter plot
plt.scatter(x, y, color='green', marker='o', label='Random data')

# Add labels and title
plt.xlabel('X-axis')
plt.ylabel('Y-axis')
plt.title('Scatter plot: Random data')

# Add a legend
plt.legend()

# Show the plot
plt.show()
```



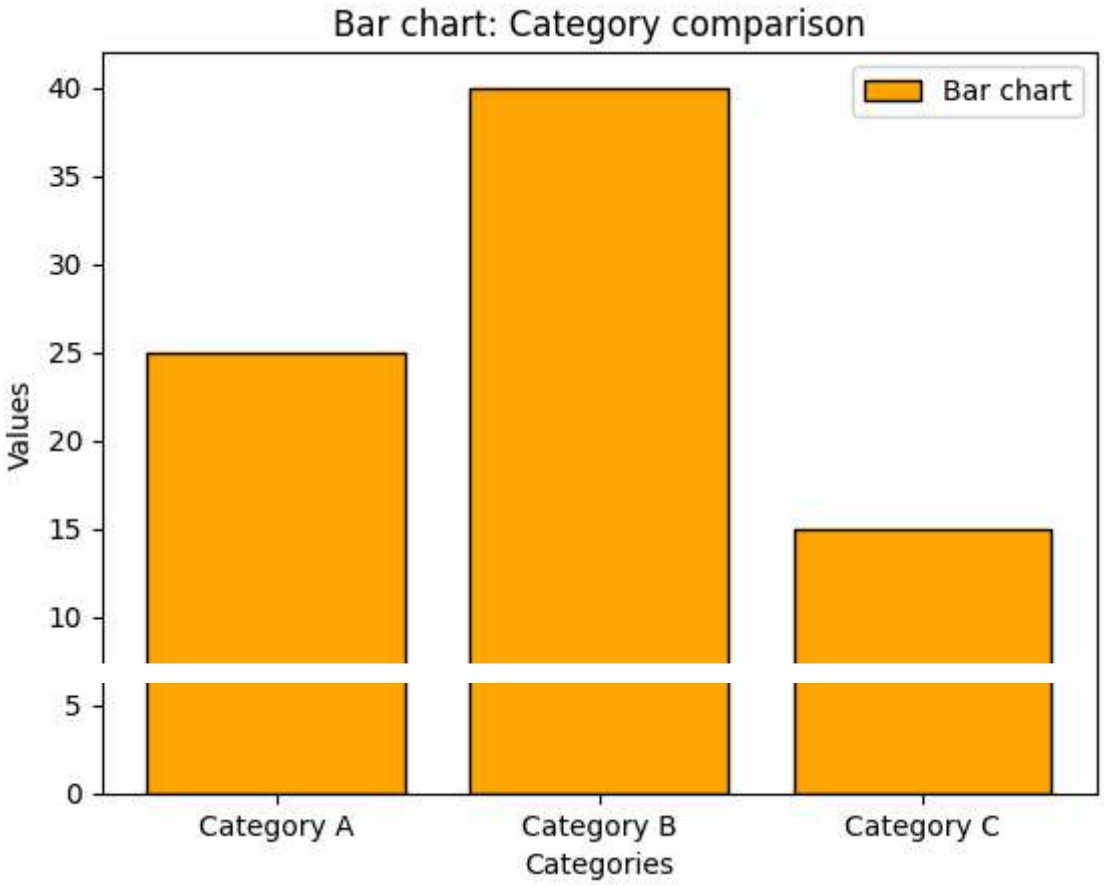
Bar Chart:

A bar chart is a common way to represent categorical data.

- It uses rectangular bars of lengths proportional to the values they represent.
- This chart is useful for comparing the sizes or frequencies of different categories.

```
import matplotlib.pyplot as plt
import numpy as np

# Generate data for a bar chart
categories = ['Category A', 'Category B', 'Category C']
values = [25, 40, 15]

# Create a bar chart
plt.bar(categories, values, color='orange', edgecolor='black', label='Bar ch

# Add labels and title
plt.xlabel('Categories')
plt.ylabel('Values')
plt.title('Bar chart: Category comparison')

# Add a legend
plt.legend()
```

```
# Show the plot
plt.show()
```



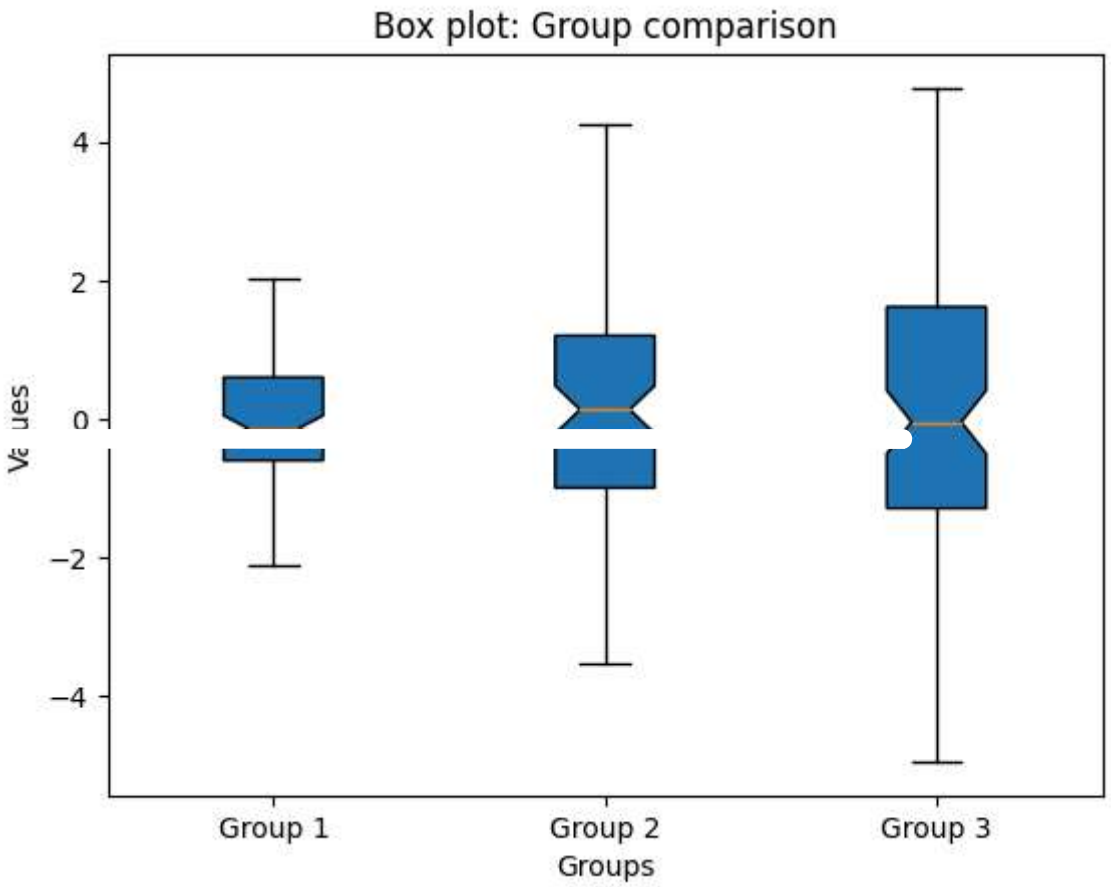
Box Plot:

A box plot (box-and-whisker plot) provides a graphical summary of the distribution of a dataset.

- It displays the minimum, first quartile, median, third quartile, and maximum values.
- This plot is useful for identifying outliers and understanding the spread and central tendency of the data.

```
import matplotlib.pyplot as plt
import numpy as np

# Generate data for a box plot
data = [np.random.normal(0, 1, 100), np.random.normal(0, 1.5, 100), np.rando

# Create a box plot
plt.boxplot(data, labels=['Group 1', 'Group 2', 'Group 3'], patch_artist=Tru

# Add labels and title
plt.xlabel('Groups')
plt.ylabel('Values')
```

```
plt.title('Box plot: Group comparison')

# Show the plot
plt.show()
```

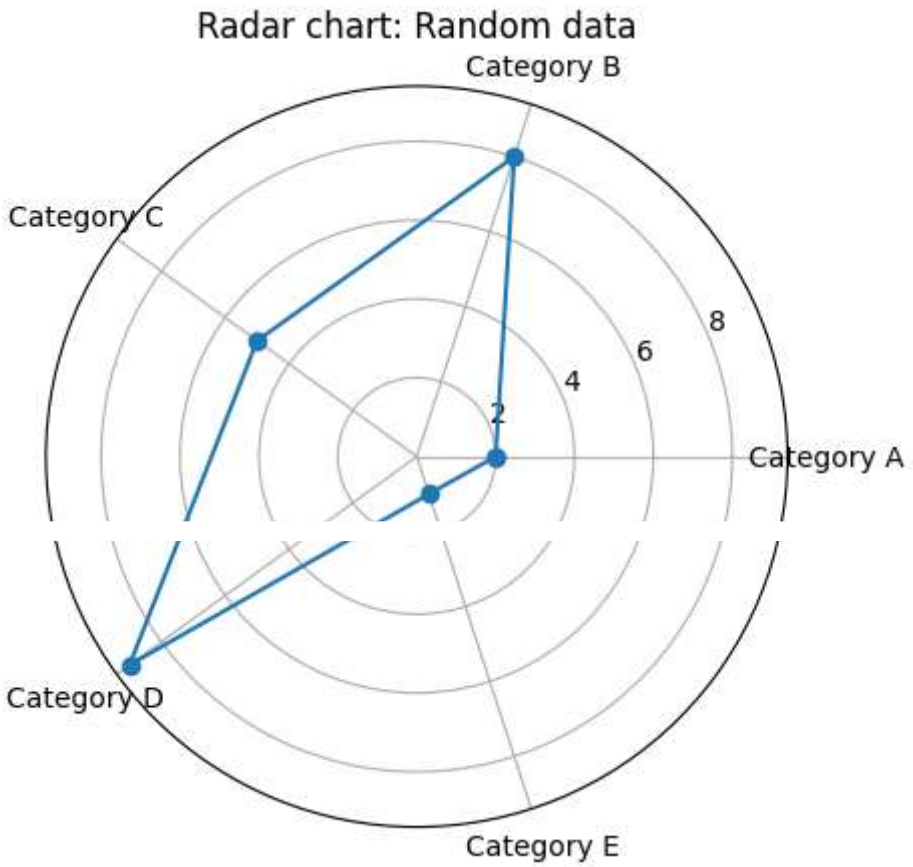


Radar Chart (Spider Chart):

A radar chart is a graphical method of displaying multivariate data in the form of a two-dimensional chart with three or more quantitative variables represented on axes starting from the same point.

```
import matplotlib.pyplot as plt
import numpy as np

# Generate random data for a radar chart
categories = ['Category A', 'Category B', 'Category C', 'Category D', 'Categ
data = np.random.randint(1, 10, len(categories))

# Create a radar chart
angles = np.linspace(0, 2 * np.pi, len(categories), endpoint=False)
data = np.concatenate((data, [data[0]]))
angles = np.concatenate((angles, [angles[0]]))
plt.polar(angles, data, marker='o')

# Add labels
```

```
plt.thetagrids(np.degrees(angles[:-1]), labels=categories)

# Show the plot
plt.title('Radar chart: Random data')
plt.show()
```



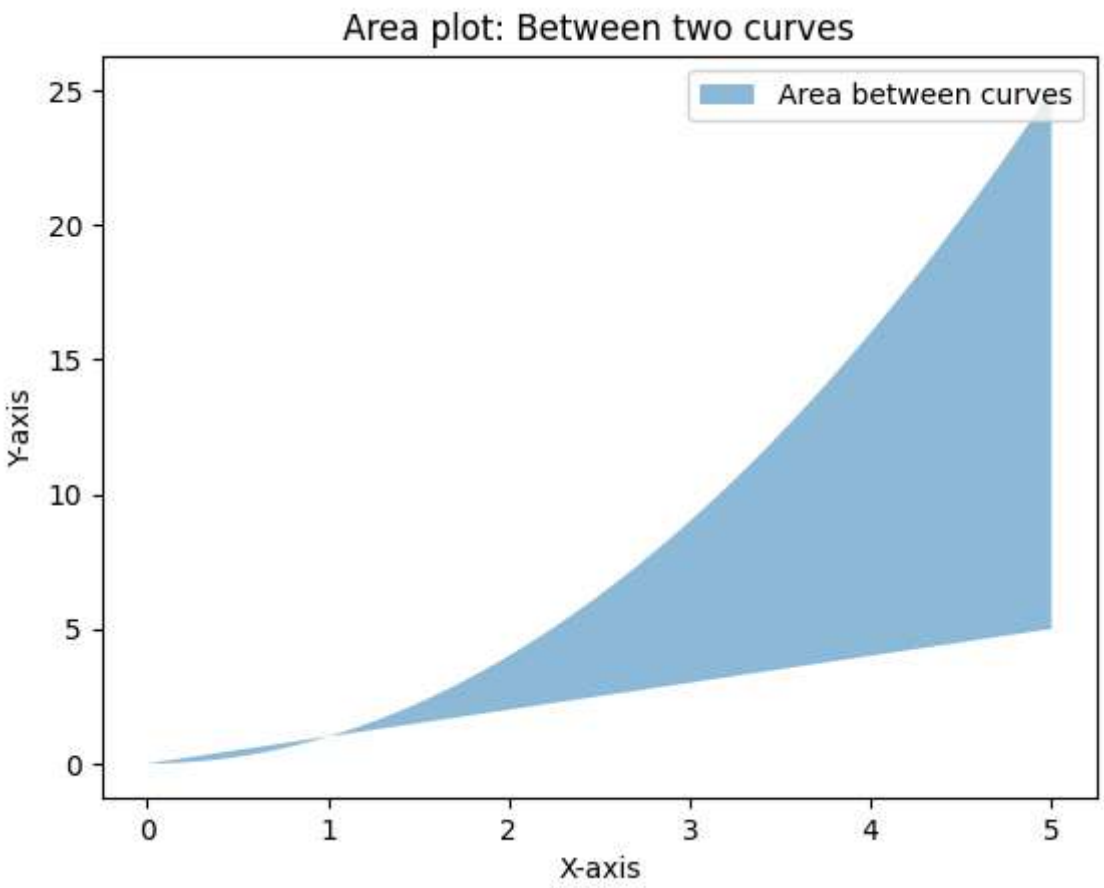
Area Plot:

An area plot is used to represent the cumulative sum of quantities over a continuous interval.

- It is effective for illustrating the contribution of each variable to the total.

```
import matplotlib.pyplot as plt
import numpy as np

# Generate data for an area plot
x = np.linspace(0, 5, 100)
y1 = x
y2 = x**2

# Create an area plot
plt.fill_between(x, y1, y2, alpha=0.5, label='Area between curves')

# Add labels and title
plt.xlabel('X-axis')
plt.ylabel('Y-axis')
```

```
plt.title('Area plot: Between two curves')

# Add a legend
plt.legend()

# Show the plot
plt.show()
```



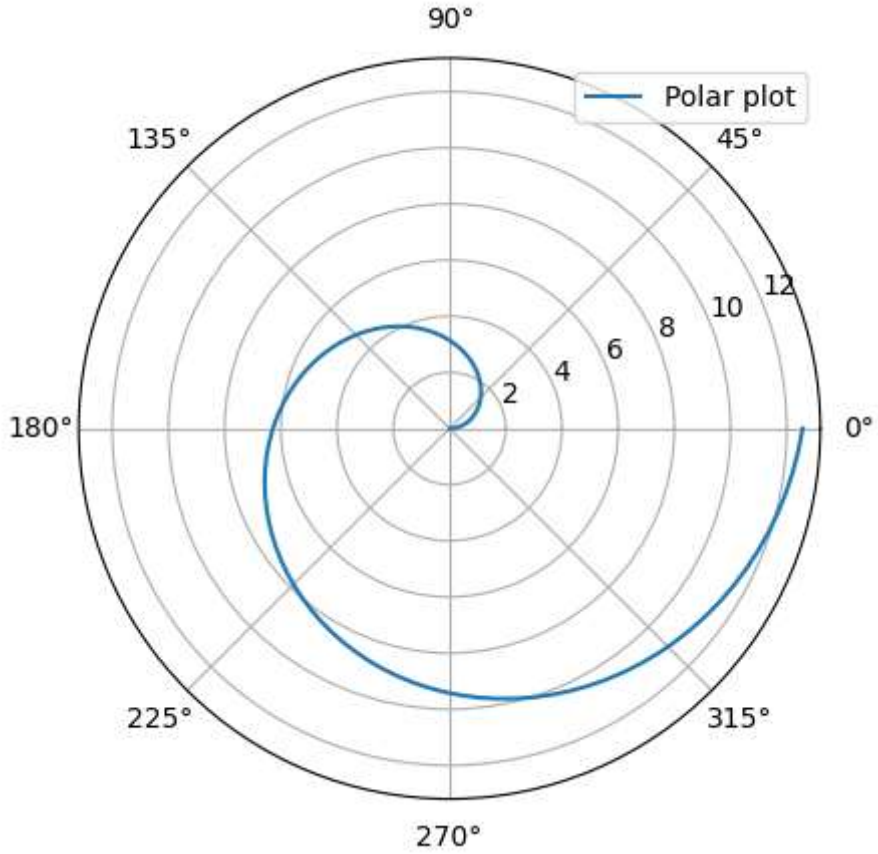
Polar Plot:

A polar plot represents data in polar coordinates.

- It is useful for visualizing cyclic patterns and relationships.

```
import matplotlib.pyplot as plt
import numpy as np

# Generate data for a polar plot
theta = np.linspace(0, 2 * np.pi, 100)
r = theta * 2

# Create a polar plot
plt.polar(theta, r, label='Polar plot')

# Add a legend
plt.legend()
```

```
# Show the plot
plt.show()
```



Tree Map:

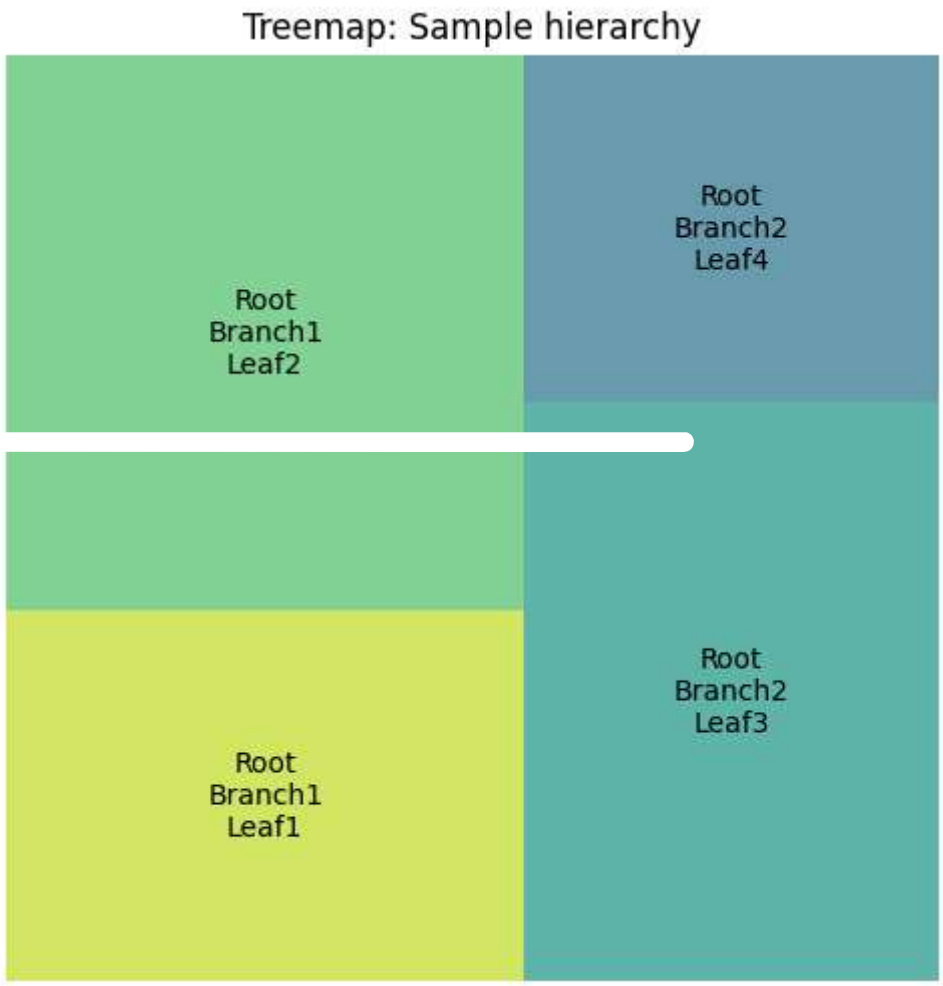
A treemap is an effective way to visualize hierarchical data structures.

- It provides an overview of the entire hierarchy while allowing users to easily compare the sizes of different branches and identify patterns within the data.

```
#!pip install squarify
import matplotlib.pyplot as plt
import squarify

# Sample hierarchical data
data = {
    'Root': {
        'Branch1': {'Leaf1': 20, 'Leaf2': 30},
        'Branch2': {'Leaf3': 25, 'Leaf4': 15}
    }
}

# Function to flatten hierarchical data
def flatten_hierarchy(node, parent=''):
    items = []
    for key, value in node.items():
```

```
            if isinstance(value, dict):
                items.extend(flatten_hierarchy(value, parent=f'{parent}\n{key}'
            else:
                items.append((parent, key, value))
        return items

    # Flatten the hierarchical data for treemap plotting
    flat_data = flatten_hierarchy(data)

    # Plotting the treemap
    fig, ax = plt.subplots(figsize=(6, 6))
    squarify.plot(sizes=[item[2] for item in flat_data], label=[f'{item[0]}\n{it

    # Remove axis labels and ticks for better aesthetics
    plt.axis('off')

    # Add a title
    plt.title('Treemap: Sample hierarchy')

    # Show the plot
    plt.show()
```

```
Collecting squarify
  Downloading squarify-0.4.3-py3-none-any.whl (4.3 kB)
Installing collected packages: squarify
Successfully installed squarify-0.4.3
```

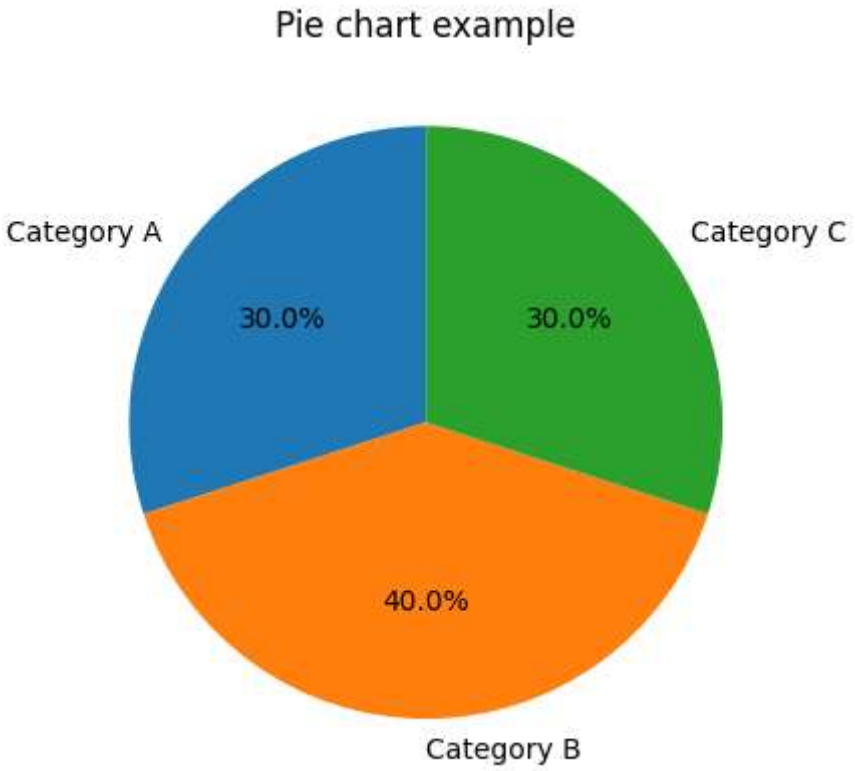


Pie Chart:

A pie chart is a circular statistical graphic that is divided into slices to illustrate numerical proportions.

- Each slice represents a proportionate part of the whole, and the entire circle represents 100%.
- These charts are commonly used to visualize the distribution of a categorical variable or the relative sizes of different components within a whole.

```
import matplotlib.pyplot as plt

# Sample data
values = [30, 40, 30]
labels = ['Category A', 'Category B', 'Category C']

# Create a pie chart
plt.pie(values, labels=labels, autopct='%1.1f%%', startangle=90)

# Add a title
plt.title('Pie chart example')

# Display the chart
plt.show()
```



Double-click (or enter) to edit

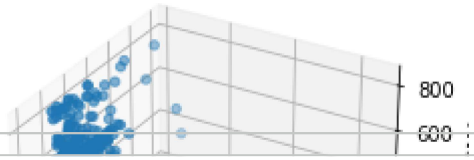
Matplotlib for 3D Visualization

Matplotlib for 3D visualization refers to the extension of the Matplotlib library's capabilities to create three-dimensional plots and visualizations in Python.

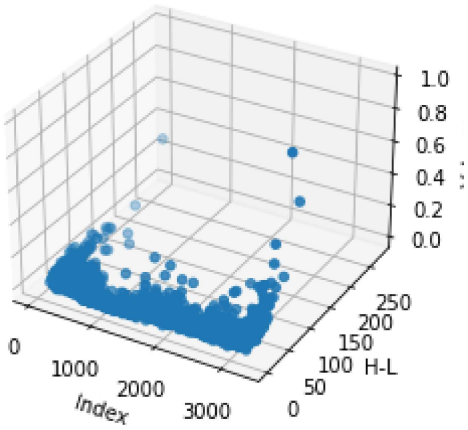
Key Features and Capabilities:

- **Creation of 3D axes:** Matplotlib enables the creation of 3D axes object for visualizing data in three dimensions.
- **Scatter plots, line plots, and surfaces:** Users can generate 3D scatter plots, line plots, and surfaces to depict relationships in three-dimensional space.
- **Customization and labeling:** Matplotlib offers options for customizing the appearance and labeling of 3D plots, enhancing visual interpretability.
- **Integration with Matplotlib's ecosystem:** The 3D visualization features seamlessly integrate with Matplotlib's broader ecosystem, facilitating combined visualizations with other plot types.

```
import pandas as pd
import matplotlib.pyplot as plt

# Read the CSV file into a Pandas DataFrame, parsing dates
df = pd.read_csv('ADANIPORTS.csv', parse_dates=True)

# Calculate the difference between High and Low prices
df['H-L'] = df.High - df.Low

# Calculate the 100-day Moving Average (100MA) of the Close prices
df['100MA'] = df['Close'].rolling(100).mean()

# Create a 3D scatter plot
ax = plt.axes(projection='3d')
ax.scatter(df.index, df['H-L'], df['100MA'])

# Set labels for each axis
ax.set_xlabel('Index')
ax.set_ylabel('H-L')
ax.set_zlabel('100MA')

# Display the 3D scatter plot
plt.show()
```




```
# Create a 3D axes object for plotting
ax = plt.axes(projection='3d')

# Scatter plot in 3D with index, 'H-L', and 'Volume'
ax.scatter(df.index, df['H-L'], df['Volume'])

# Set labels for the x, y, and z axes
ax.set_xlabel('Index')
ax.set_ylabel('H-L')
ax.set_zlabel('Volume')

# Display the 3D scatter plot
plt.show()
```



Assignment